

WEEK 4 • DAY 2

Document Pipelines & Semantic Search

From Extracted Text to Something Your AI Can Actually Search

1-Hour Lecture Session — Concepts & Architecture (No Live Platform Demo)

Rajesh Pasham

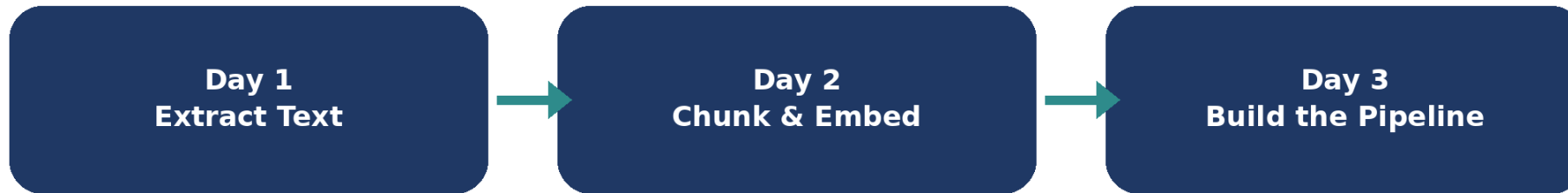
Program Context

How today's session fits the wider 8-week delivery

Session Type	Concepts and architecture lecture — no live Palantir platform work today
Platform	Individual Palantir Developer Tier accounts, used from Day 3 onward
This Week's Lab (Day 3)	Insurance Document Intelligence Pipeline — chunking, embedding, and searching real claim and policy documents

Where Today Fits

From readable text to searchable, linkable content



Day 1 got you a page of extracted text. That's not yet useful to an AI workflow at scale — a whole claim file is too long to hand an LLM every time, and exact keyword matching misses anything phrased differently. Today is about the two things that fix that: breaking text into meaningful pieces, and finding the right pieces by meaning, not just words.

Today's Roadmap

Three topic groups, building toward Wednesday's document pipeline

1 **Chunking & Embeddings**

Breaking text into pieces an embedding model can handle, and turning each piece into a searchable vector

2 **Vector Search & Hybrid Retrieval**

Finding the right chunk by meaning, combining multiple retrieval methods, and choosing full documents vs. chunks

3 **Trust & Quality**

Measuring whether retrieval actually works, and showing your source so people can check it

By the End of Today

The vocabulary Wednesday's document pipeline lab assumes



Explain why documents get chunked before they're embedded

And what happens if a chunk is too large for the embedding model you chose



Distinguish keyword search from vector search from hybrid retrieval

And know why combining methods usually beats picking just one



Choose between giving an LLM the full document or just relevant chunks

A real, configurable trade-off, not just a theoretical one



Explain how retrieval quality gets measured, and why citations matter

So a wrong answer is traceable, and a good pipeline can be proven to work

TOPIC 1

Chunking & Embeddings

Breaking text into pieces an embedding model can handle, and turning each piece into a searchable vector

Why Chunk At All

Two separate reasons, both practical



Embedding models have a maximum input length

Many models accept only 512 tokens at a time — a full claim file simply won't fit in one pass



Smaller pieces are more semantically distinct during search

A chunk about “water damage to the kitchen” is easier to match precisely than a search across an entire 40-page claim file that also covers policy terms and payment history

The Chunk String Board

Chunking, concretely: one board in Pipeline Builder

What It Does

Breaks up a larger piece of text into smaller pieces — the mechanical step behind “chunking”

Expression

The text column you want to split — typically your Day 1 extracted-text output

Chunk Size

The approximate number of characters in each chunk — this is the number your embedding model's token limit ultimately constrains

The Result

Each chunk gets its own associated Ontology object, linked back to the original document — the same Link Type discipline from Week 1, now applied to pieces of a document instead of whole records

Choosing an Embedding Model

Not a purely technical choice — it shapes your chunking strategy



Token limits work backward into chunk size

Most non-Ada embedding models accept only 512 tokens — your Chunk Size setting from the previous slide has to respect whatever limit your chosen model has



Symmetric vs. asymmetric matters for how you'll search

A model built for query-to-passage retrieval (like the models behind MS MARCO) fits a use case that starts from a short question; a model like Ada fits a use case that starts from a chunk and searches for similar chunks



Generic models can underperform on domain-specific language

A general-purpose embedding model trained on broad web text may retrieve less precisely against dense insurance or medical terminology than expected — worth testing before committing

TOPIC 2

Vector Search & Hybrid Retrieval

Finding the right chunk by meaning, combining multiple retrieval methods, and choosing full documents vs. chunks

From Chunks to Searchable Ontology Objects

Chunking and embedding, end to end



Each chunk becomes its own Ontology object, carrying a vector property that holds its embedding — this is what makes a chunk searchable by meaning, not just by exact text.

Vector Search: Nearest Neighbors in Embedding Space

Finding chunks by meaning, not by matching words



A query gets embedded the same way a chunk does

Then the search finds the chunks whose vectors sit closest to the query's vector — “nearest neighbors” in that embedding space

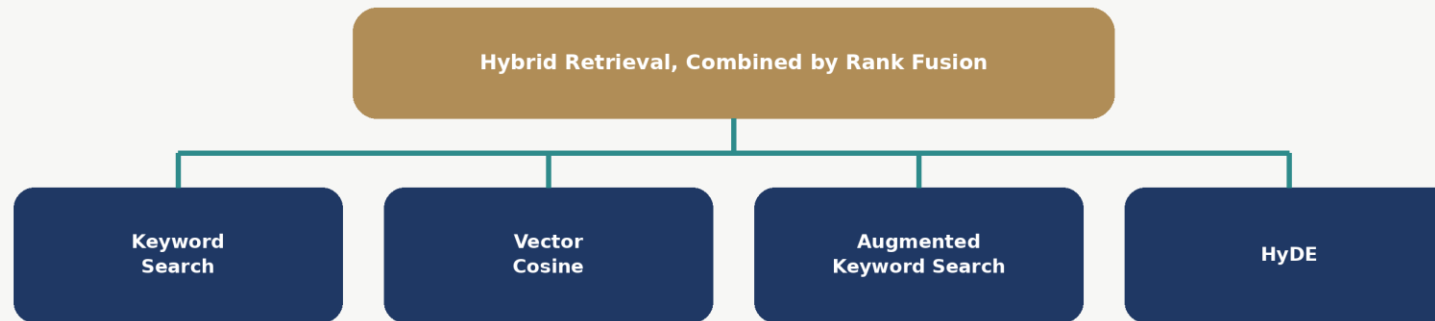


HyDE bridges the gap when a query and a chunk don't naturally look alike

Hypothetical Document Embeddings: generate a plausible answer with an LLM first, embed that instead of the raw query, then search — useful because a chunk and an answer are more alike than a chunk and a question

Hybrid Retrieval: Combining Multiple Methods

No single method wins every time — rank fusion combines their results



Keyword search catches exact terms vector search might miss; vector search catches meaning keyword search can't see. Combining results from more than one method, via rank fusion, is usually more reliable than trusting any single approach.

Full Document vs. Relevant Chunks

A real, configurable choice — not just a theoretical trade-off

Full Document Text Mode

Gives the LLM the entire document as context — complete, but expensive in tokens, and impractical for a long claim file

Relevant Chunks Mode

Performs a semantic search over the document and returns only the K most relevant chunks — efficient, but only as good as the retrieval that selected those chunks

The Practical Rule

Short, single documents can often use full-document mode. Anything long, or any workflow searching across many documents, needs chunk-based retrieval to stay both accurate and affordable

TOPIC 3

Trust & Quality

Measuring whether retrieval actually works, and showing your source so people can check it

Retrieval Quality Evaluation

The same evaluation discipline from Week 2, now aimed at retrieval specifically



An evaluation suite can compare retrieval techniques directly

Benchmarking keyword search, vector cosine, augmented keyword search, and HyDE against the same set of test questions, to see which retrieves the most relevant chunks



A wrong answer is often a retrieval failure, not a reasoning failure

If an AIP-associated tool fails to answer something that should be in the document corpus, check first whether the right context was ever retrieved — the LLM can only reason over what it was actually given

Citation & Provenance Tracking

Letting a person check the source, not just trust the answer



Every retrieved chunk should trace back to its source document

The same citation discipline from Week 2's groundedReferences field, now anchored to a specific chunk of a specific file rather than an Ontology object



Surfacing the original document alongside the answer builds trust

Rendering the source PDF next to a retrieved chunk lets a claims handler or underwriter cross-check the answer against the actual document, not just take the AI's word for it

Why This Matters for Day 3's Lab

Every concept today becomes a real pipeline step on Wednesday

Wednesday's Insurance Document Intelligence Pipeline chunks real claim forms and medical records, embeds each chunk, and links it back to the Claim and Insurance Policy objects from Week 1. The retrieval method you choose, and whether you cite your sources, directly determines whether the resulting AI workflow can be trusted in production — not just whether it runs.

Common Pitfalls

Patterns worth avoiding before Wednesday's lab

Chunking without checking the embedding model's token limit

A chunk sized larger than the model accepts gets silently truncated or rejected — check the limit first, then set chunk size

Assuming vector search alone is always better than keyword search

Vector search can miss an exact term match that a simple keyword search would have caught — hybrid retrieval exists because neither method wins every time

Defaulting to full-document context out of convenience

It works for a short policy summary and fails, expensively, for a 40-page claim file — match the mode to the document length

Treating retrieval as untestable

A retrieval-specific evaluation suite, benchmarking multiple methods against real questions, is exactly as buildable as the workflow evaluation suites from Week 2

Recap & What's Next

What We Covered Today

- Chunking strategy and the Chunk String board
- Choosing an embedding model, and its token limits
- Vector search, HyDE, and hybrid retrieval via rank fusion
- Full document vs. relevant chunks
- Retrieval quality evaluation and citation tracking

Day 3 — Self-Paced Lab

- Insurance Document Intelligence Pipeline
- Extract, chunk, and embed real claim and policy documents
- Link chunks back to your Week 1 Ontology objects
- Build and test a hybrid retrieval workflow



Questions

Rajesh Pasham